Fitness & Nutrition Tracker

Project Report

A Dynamic Web Application using HTML, CSS & JavaScript

Submitted by:

Name	Prateek Goel
Matriculation Number	401054851
Course	MSc in SAP Engineering & Analytics
University	Hochschule Fresenius University of Applied Sciences, Heidelberg
Date	March 2026

1. Title & Objective

1.1 Application Title

Fitness & Nutrition Tracker

1.2 What Does the App Do?

The Fitness & Nutrition Tracker is a web-based application built entirely using HTML, CSS, and JavaScript — with no external libraries or frameworks. The app allows a user to enter their personal body details such as name, age, gender, weight, height, activity level, and fitness goal. Based on this information, the app calculates and displays:

- BMI (Body Mass Index) — to assess if the user is underweight, normal, overweight, or obese
- BMR (Basal Metabolic Rate) — the number of calories the body needs at complete rest
- Daily Calorie Burn — BMR adjusted for activity level
- Target Calories — adjusted based on the user's goal (lose weight, maintain, or gain muscle)
- Macronutrient targets — daily protein, carbohydrates, and fat intake in grams

After viewing the results, the user can log food items in the Daily Meal Logger. Each logged item is added to a dynamic table and the running totals for calories, protein, carbs, and fat are updated live. The app also shows how much of the daily target remains and gives an over/under target warning with colour-coded messages. A Reset button allows the user to clear everything and start fresh.

1.3 Purpose

The purpose of this app is to help individuals track their daily calorie and macronutrient intake based on their specific body parameters and fitness goal. Instead of using a generic calorie target, this app personalises the results using scientifically established formulas — making it more accurate and useful than a simple calculator.

1.4 Target Users

The application is designed for:

- Fitness beginners who want to understand how many calories they should eat
- People trying to lose weight, maintain weight, or gain muscle
- Anyone who wants to track their daily meals against a personalised nutrition target
- Students and developers learning how to build dynamic JavaScript applications

2. Functionality & Features

2.1 Input Form — Collecting Body Details

The first section of the app is the input form. It collects all the information needed to calculate the user's personalised nutrition targets. The form contains:

- Text input for the user's name
- Number input for age
- Dropdown (select) for gender — Male or Female
- Number input for weight in kilograms
- Number input for height in centimetres
- Dropdown for activity level — 4 options from Sedentary to Very Active
- Dropdown for fitness goal — Weight Loss, Maintain Weight, or Gain Muscle
- A green Calculate button to trigger all calculations
- A grey Reset Everything button to clear the form

All dropdown fields show a placeholder option by default (e.g. --Select Gender--) so the user must actively choose before calculating.

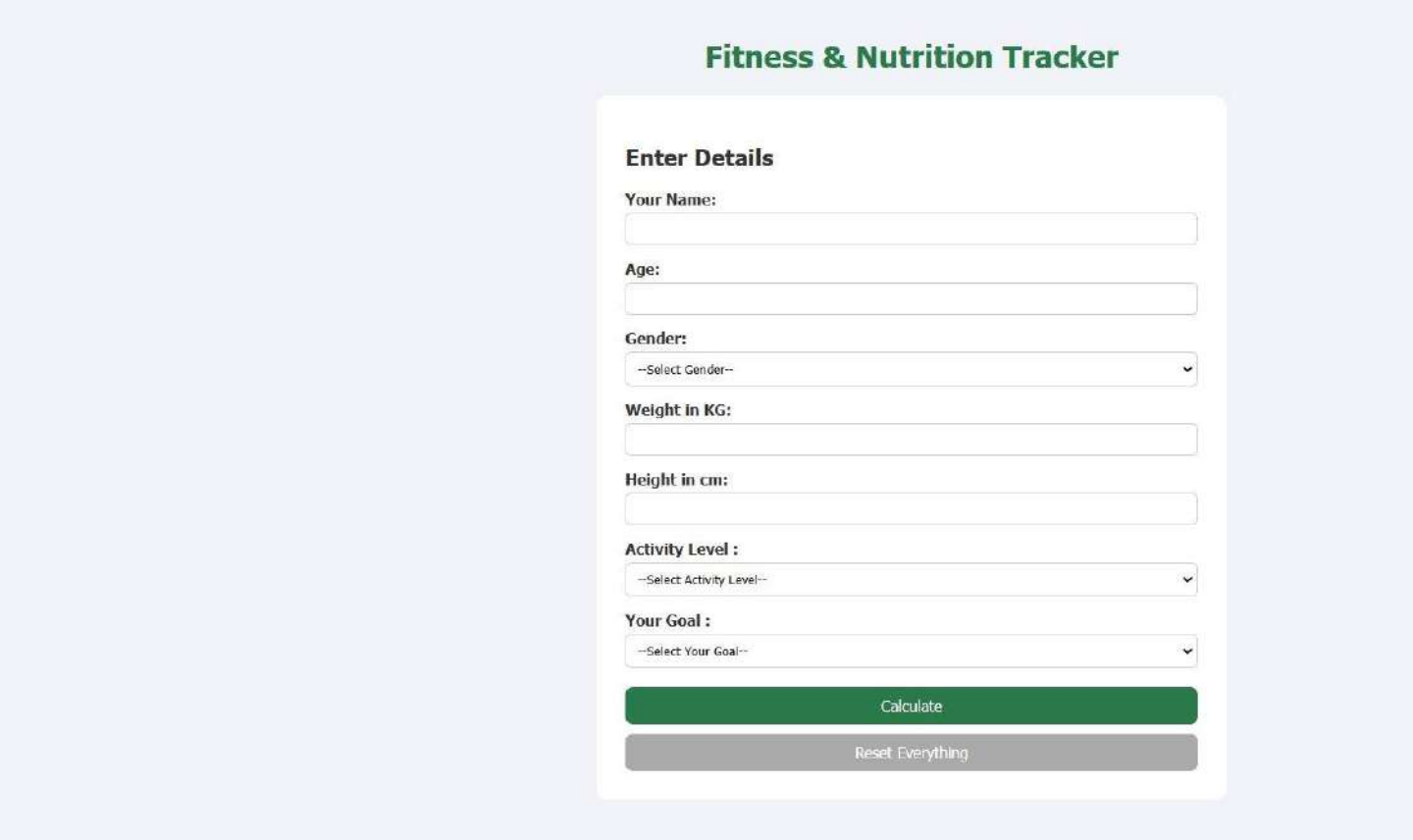

Figure 1: Empty Input Form with placeholder dropdowns

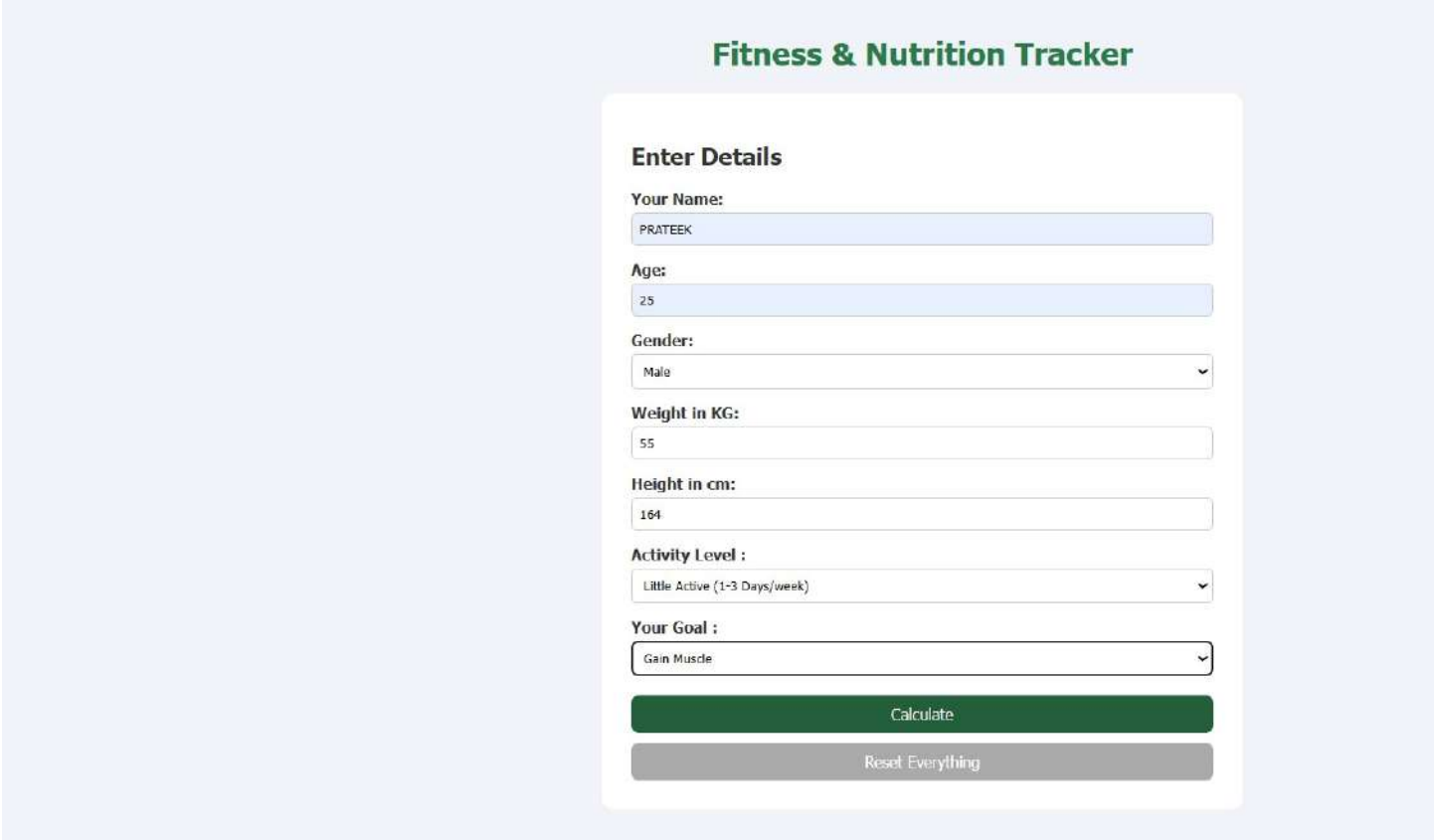

Figure 2: Input Form filled in with sample data (Prateek, 25 years, Male, 55kg, 164cm, Lightly Active, Gain Muscle)

2.2 Input Validation

Before any calculation takes place, the app validates all inputs. This is an important feature that protects the app from crashes caused by empty or invalid data. Two validation checks are performed:

- Check 1: If name, age, weight, height, gender, activity, or goal is empty, an alert appears and the calculation stops.
- Check 2: If age, weight, or height is zero or negative, a second alert appears and the calculation stops.

Only when all inputs pass both checks does the app proceed to calculate results.

2.3 Results Section — Calculated Output

After the user clicks Calculate, a results section appears below the form (it is hidden by default and becomes visible using JavaScript). The results section displays:

- A personalised greeting using the user's name
- BMI value and category (Underweight / Normal Weight / Overweight / Obese)
- BMR in kcal — calories needed at complete rest
- Daily Calorie Burn — BMR multiplied by activity multiplier

- Target Calories — adjusted based on goal
- Daily targets for Protein, Carbohydrates, and Fat in grams

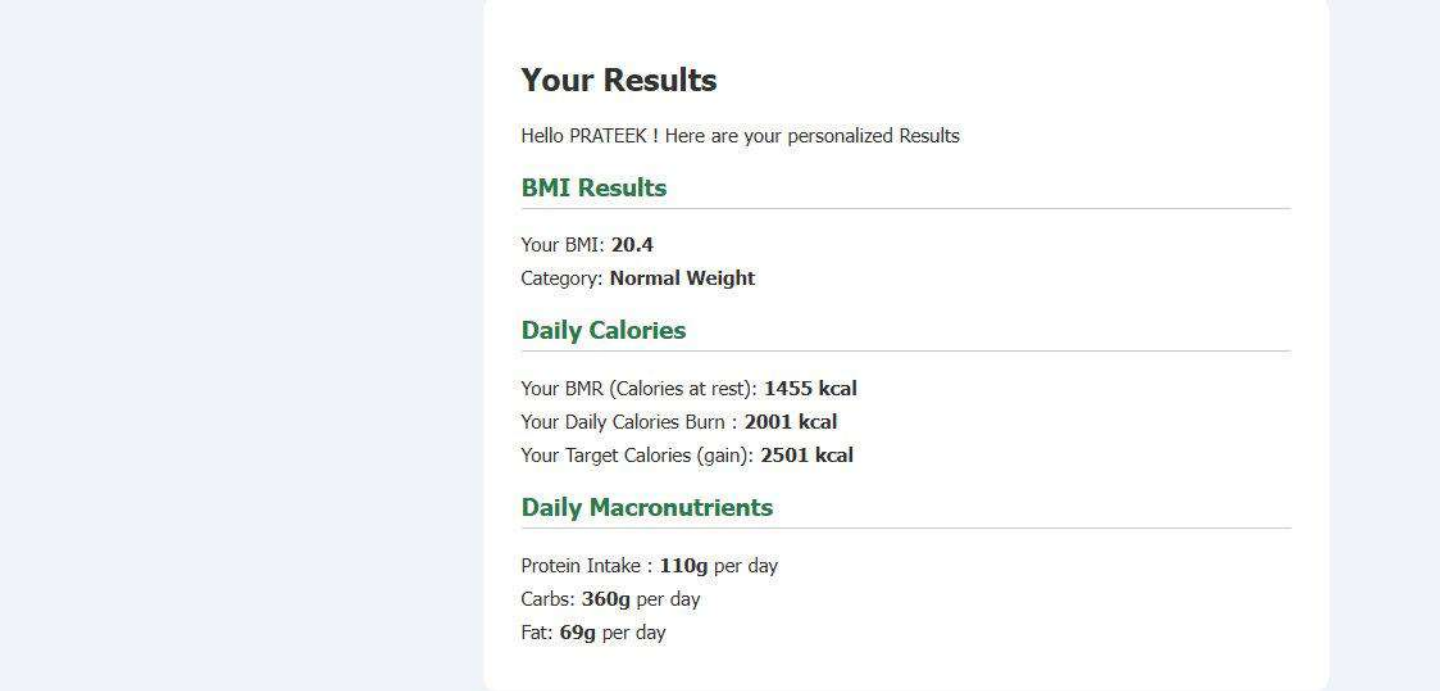

Figure 3: Results section showing BMI, Daily Calories, and Macronutrients for Prateek

2.4 Daily Meal Logger

The meal logger is the most interactive feature of the app. It appears below the results section after calculation. It allows the user to:

- Enter a food item name with its calorie and macronutrient values
- Click Add Meal to log the food — it is immediately added to a dynamic table
- See running daily totals for all macros update instantly after each addition
- Delete any meal from the table using the red Delete button — totals update automatically
- See how much of the daily target remains after logged meals

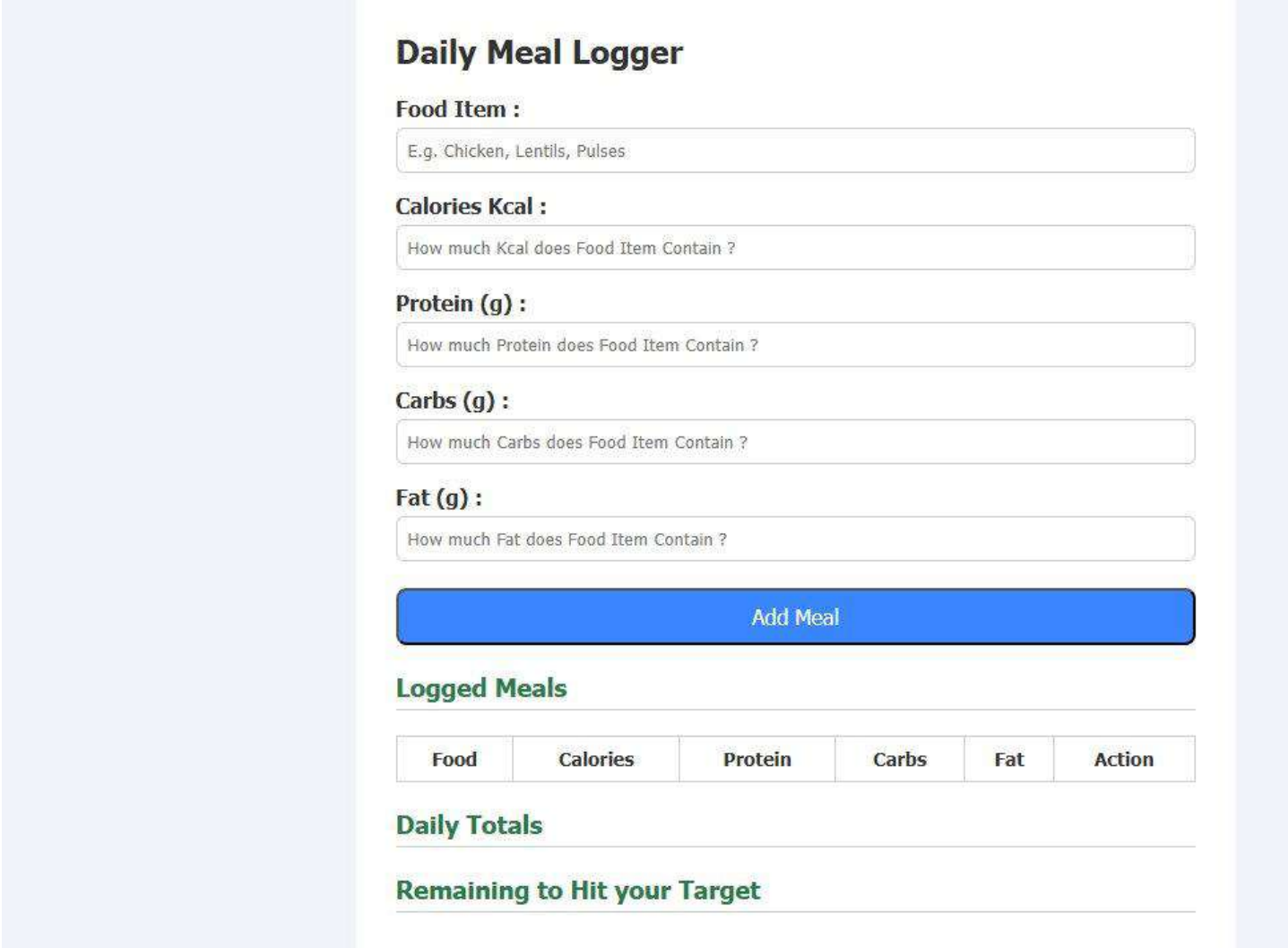

Figure 4: Meal Logger section with empty table, ready to log food

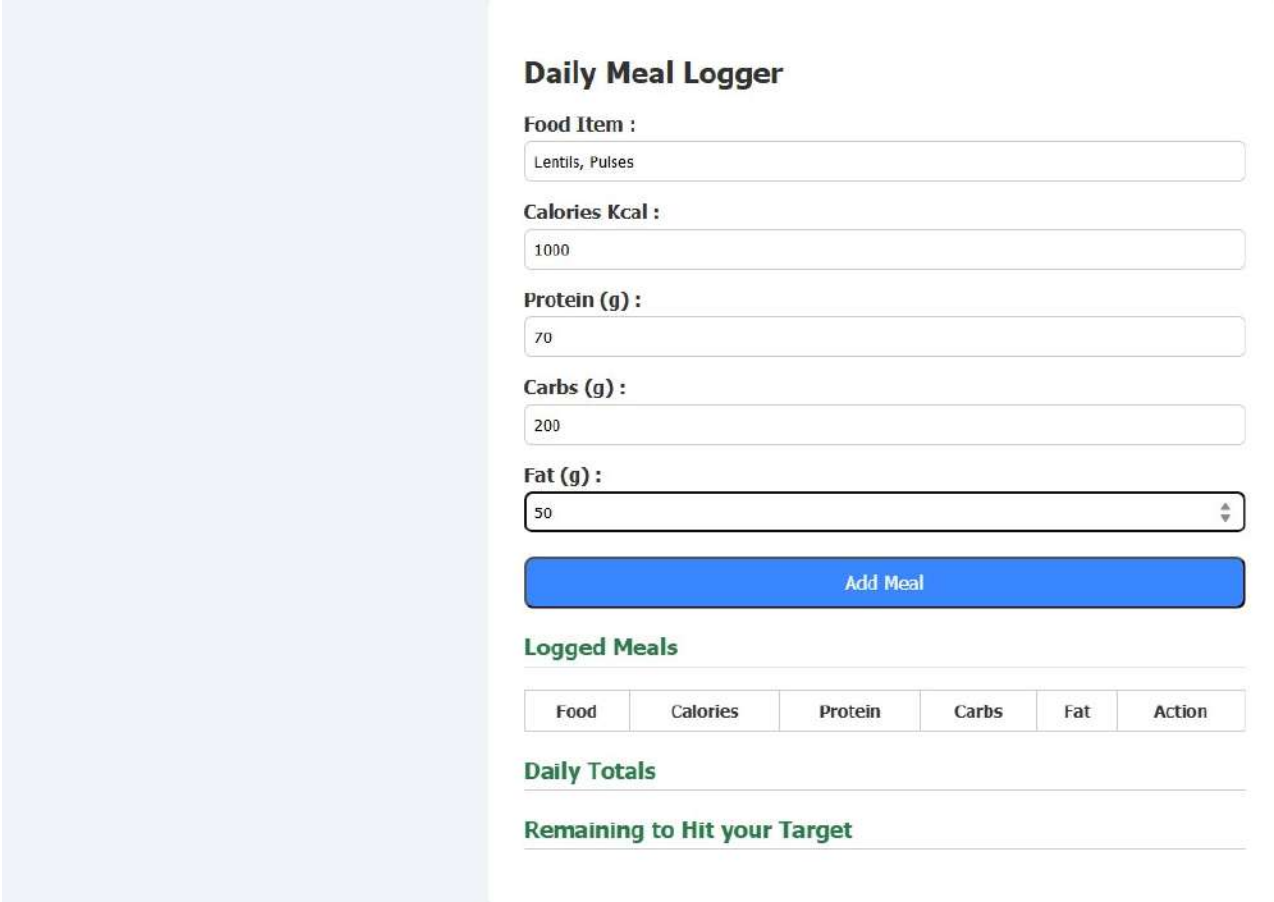

Figure 5: Meal Logger with food item (Lentils, Pulses) being entered before clicking Add Meal

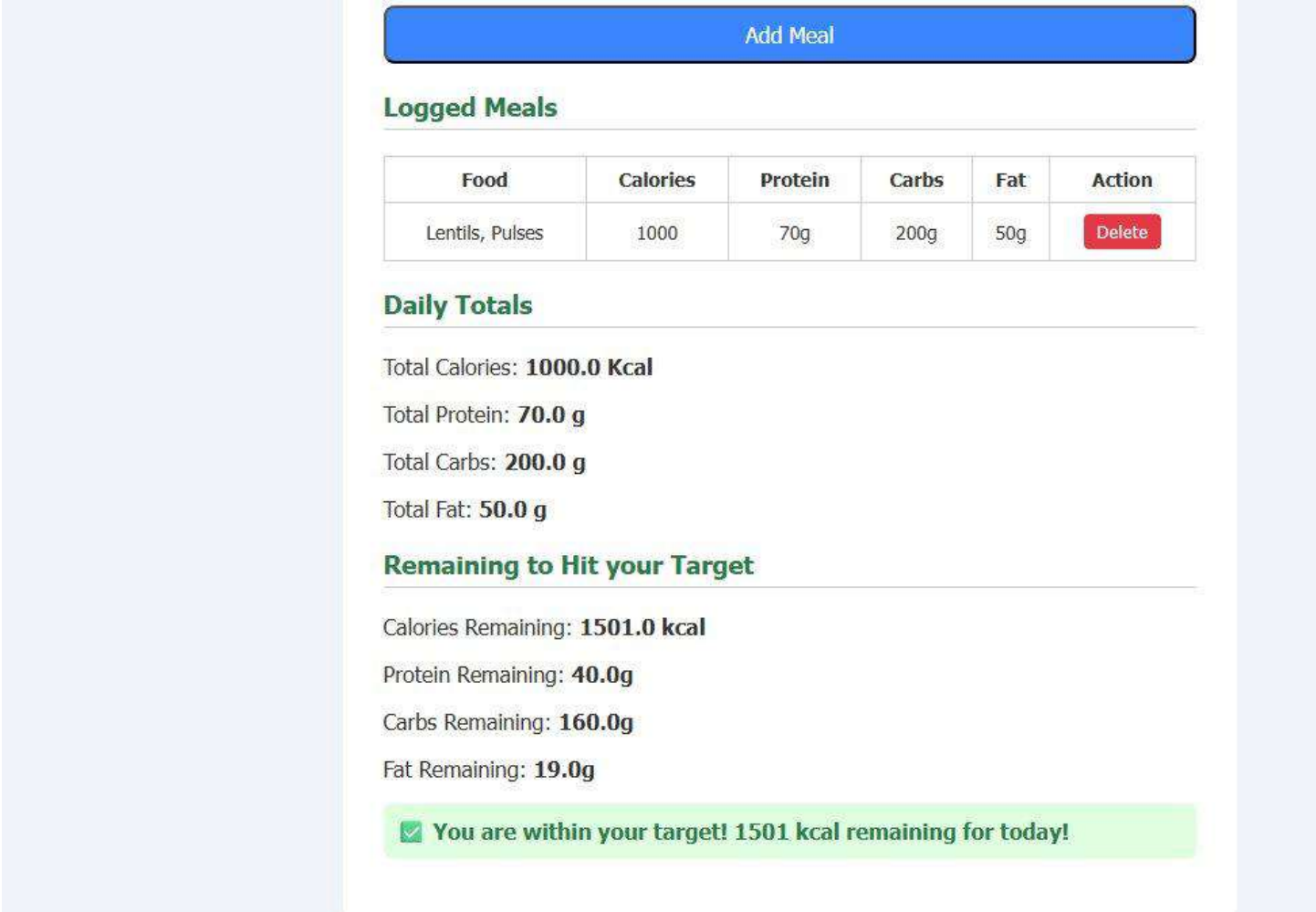

Figure 6: After clicking Add Meal — table shows Lentils/Pulses, totals updated, green warning shown

2.5 Over / Under Target Warning

After each meal is added or deleted, a colour-coded warning message is shown at the bottom of the meal logger:

Situation	Colour	Message
More than 100 kcal over target	**Red**	Exceeded calorie target warning
More than 100 kcal remaining	**Green**	Within target, kcal remaining shown
Within 100 kcal of target	**Yellow**	Perfect — daily calorie target hit!

Logged Meals

Food	Calories	Protein	Carbs	Fat	Action
Chicken	3000	110g	400g	100g	Delete

Daily Totals

Total Calories: **3000.0 Kcal**

Total Protein: **110.0 g**

Total Carbs: **400.0 g**

Total Fat: **100.0 g**

Remaining to Hit your Target

Calories Remaining: **-499.0 kcal**

Protein Remaining: **0.0g**

Carbs Remaining: **-40.0g**

Fat Remaining: **-31.0g**

⚠️ **You have exceeded your calorie target by 499 kcal! Consider lighter Meals!**

Figure 7: Red warning — Calorie target exceeded by 499 kcal

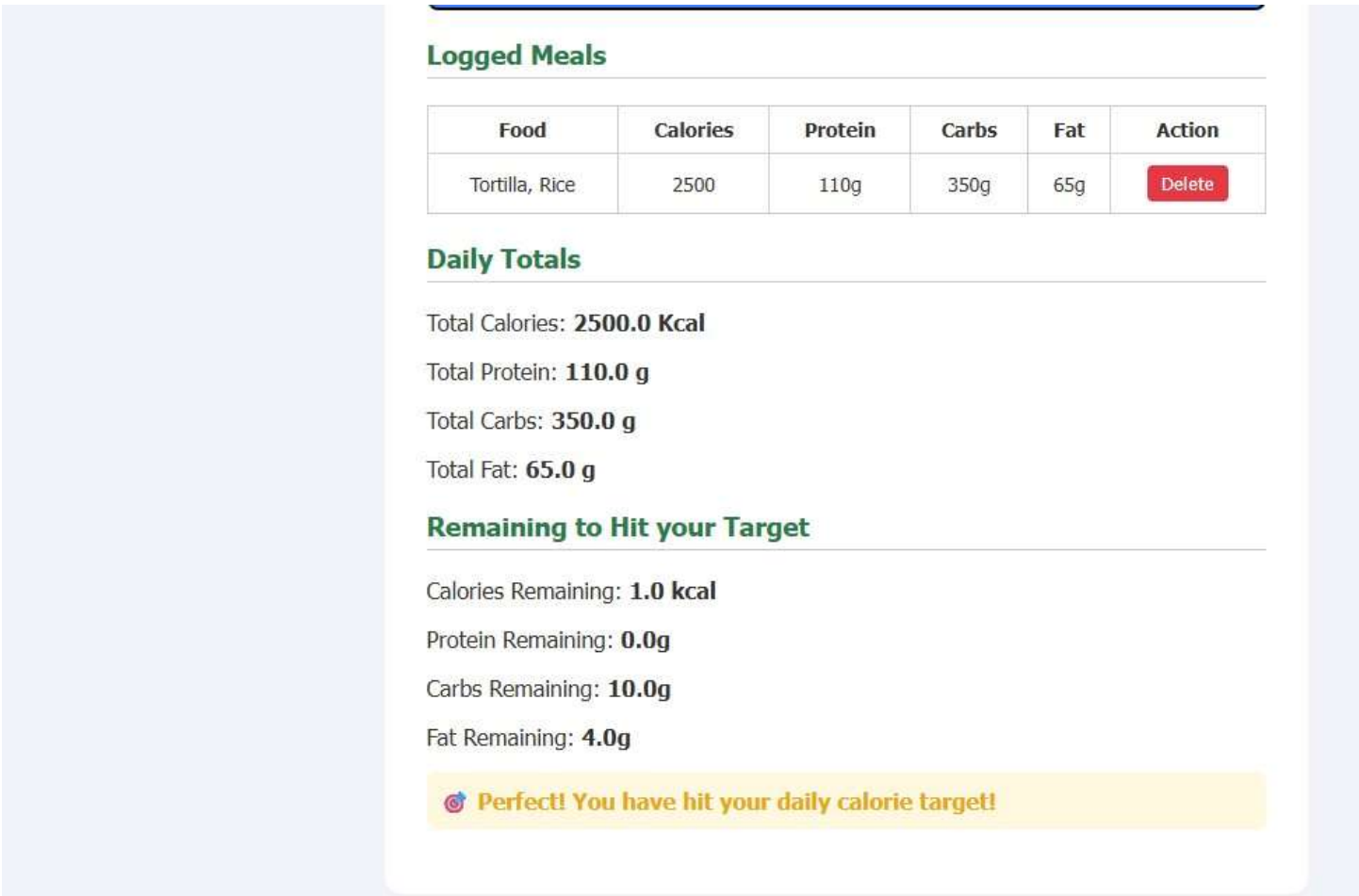

Figure 8: Yellow warning — Perfect, daily calorie target hit

2.6 Reset Button

The Reset Everything button (shown in grey below the Calculate button) allows the user to:

- Clear all input fields back to empty
- Hide the results section and meal logger section
- Reset all running totals and global variables to zero
- Clear the meal table completely

A confirmation dialog appears first asking "Are you sure you want to reset everything?" — the user can cancel if they clicked Reset by accident.

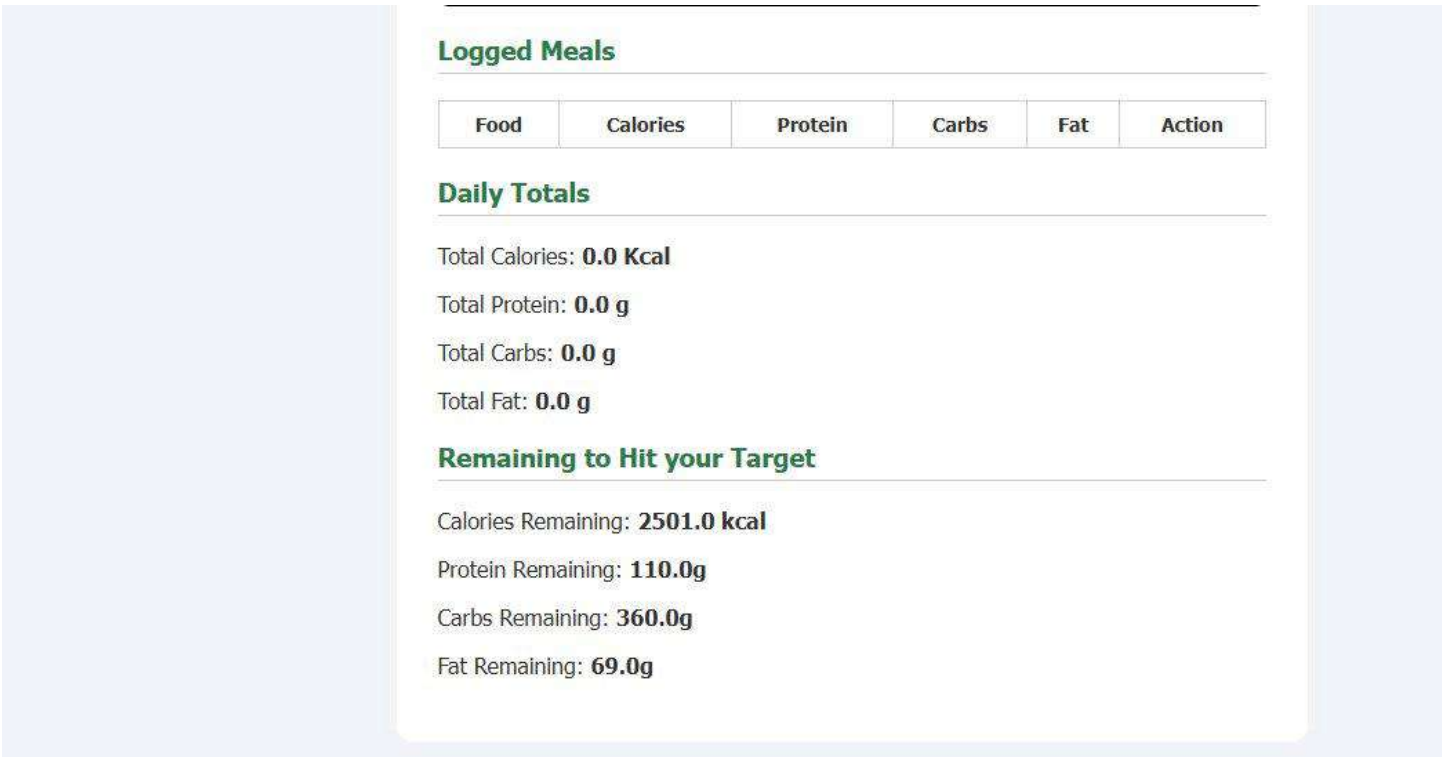

Figure 9: Meal logger showing 0.0 totals after items have been deleted

3. Code Snippets & Explanations

This section explains the key JavaScript functions and concepts used in the application. The logic behind every dynamic feature is covered here in simple terms.

3.1 How JavaScript is Structured

All JavaScript is written inside a <script> tag placed at the very bottom of the HTML file, just before </body>. This is intentional — the browser reads the file from top to bottom. By placing JavaScript at the bottom, all HTML elements are fully loaded before JavaScript tries to interact with them. If JavaScript were placed in the <head>, it would run before the HTML existed and cause errors.

3.2 Global Variables

At the very top of the JavaScript section, four global variables are declared:

```
let targetCalorie = 0;
let protein       = 0;
let carbs         = 0;
let fat           = 0;
```

These are declared globally (outside all functions) so that every function in the application can read and update them. This was a critical decision — early in development, these variables were declared inside the calculate function using let, which caused the meal logger to always see 0 because the variables were only local to the calculate function. Moving them to the top fixed the issue. This concept is explained further in Section 4 (Challenges).

3.3 Reading User Input — getElementById and .value

When the user clicks Calculate, the first thing JavaScript does is read all the values the user has typed or selected:

```
let name     = document.getElementById("name").value;
let age      = document.getElementById("age").value;
let gender   = document.getElementById("gender").value;
let weight   = document.getElementById("weight").value;
let height   = document.getElementById("height").value;
let activity = document.getElementById("activity").value;
let goal     = document.getElementById("goal").value;
```

document.getElementById("name") tells JavaScript to find the HTML element that has id="name". Adding .value at the end retrieves whatever the user typed or selected. Each input and dropdown in the HTML was given a unique id exactly so JavaScript can find it this way. Think of id as a unique address for each element on the page.

3.4 Input Validation — if/else and return

```
if(name===""||age===""||weight===""||height===""||gender===""||activity===""||goal=
==""){
    alert("Please fill in the Mandatory Details");
    return;
}
if(age <= 0 || height <= 0 || weight <= 0){
    alert("Age, Weight & Height must be greater than zero");
    return;
}
```

The === operator checks for exact equality. The || operator means OR — if any single field is empty, the first check triggers. The return keyword stops the function immediately — without it, the calculation would continue with empty values and produce incorrect results. This is called defensive programming.

3.5 BMI Calculation

```
age    = parseFloat(age);
weight = parseFloat(weight);
height = parseFloat(height);

let heightMtrs = height / 100;
let bmi = weight / (heightMtrs * heightMtrs);
bmi = bmi.toFixed(1);
```

When JavaScript reads values from input boxes, they come as text (strings) even if the user typed a number. parseFloat() converts text to a decimal number so mathematical operations work correctly. Without this, "70" + 10 would give "7010" instead of 80.

The BMI formula is: Weight (kg) divided by Height (m) squared. Since the user enters height in centimetres, we divide by 100 to convert to metres first. toFixed(1) rounds the result to 1 decimal place — so 20.456 becomes 20.5.

3.6 BMI Category — if / else if / else

```
let bmiCategory;
if (bmi < 18.5) {
    bmiCategory = "Underweight";
} else if (bmi < 25) {
    bmiCategory = "Normal Weight";
} else if (bmi < 30) {
    bmiCategory = "Overweight";
} else {
    bmiCategory = "Obese";
```

```
}
```

This chain of conditions checks the BMI value and assigns the appropriate category. Only one block runs — whichever condition is true first. The final else catches everything 30 and above. The thresholds used are the standard WHO (World Health Organisation) BMI classifications.

3.7 BMR Calculation — The Mifflin-St Jeor Formula

```
let bmr;
if(gender === "male"){
    bmr = (10 * weight) + (6.25 * height) - (5 * age) + 5;
} else {
    bmr = (10 * weight) + (6.25 * height) - (5 * age) - 161;
}
bmr = Math.round(bmr);
```

Why the Mifflin-St Jeor Formula?

BMR stands for Basal Metabolic Rate — it is the number of calories your body needs just to stay alive at complete rest (breathing, heart beating, organ function). There are several formulas available (Harris-Benedict, Mifflin-St Jeor, Katch-McArdle). The Mifflin-St Jeor formula was chosen because:

- It is widely considered the most accurate formula for the general population
- It was validated in multiple research studies and is recommended by the Academy of Nutrition and Dietetics
- It uses weight, height, age, and gender — all of which we already collect from the user

The male and female formulas differ only in the last constant (+5 for males, -161 for females) because men generally have more lean muscle mass and a higher metabolic rate than women of the same weight and height.

Math.round() rounds the BMR to the nearest whole number since calorie values are always expressed as whole numbers in nutrition.

3.8 Daily Calories & Activity Multipliers

```
let activityMulti = parseFloat(activity);
let dailyCalories = Math.round(bmr * activityMulti);
```

Why These Fixed Activity Multiplier Values?

The activity dropdown uses specific numeric values:

Sedentary (little/no exercise)	1.2	Only 20% above BMR
Lightly Active (1-3 days/week)	1.375	37.5% above BMR
Moderately Active (3-5 days/week)	1.55	55% above BMR
Very Active (6-7 days/week)	1.725	72.5% above BMR

These multipliers come from the Harris-Benedict Activity Factor table, which is a standard in exercise science and nutrition. They represent how much more energy the body burns above the resting rate depending on physical activity. These are not arbitrary numbers — they are established constants used by nutritionists and dietitians worldwide.

3.9 Adjusting for Goal

```
if(goal === "loss") {
    targetCalorie = dailyCalories - 500;
} else if (goal === "maintain") {
    targetCalorie = dailyCalories;
} else {
    targetCalorie = dailyCalories + 500;
}
```

The 500 calorie adjustment is based on the widely accepted principle that 1 pound of body fat equals approximately 3,500 calories. A daily deficit or surplus of 500 calories leads to approximately 0.5kg of weight loss or gain per week — which is considered a healthy, sustainable rate of change. This formula is used by most professional nutrition apps and dietitians.

3.10 Macronutrient Calculation

```
protein = Math.round(weight * 2);
fat     = Math.round((targetCalorie * 0.25) / 9);
carbs   = Math.round((targetCalorie - (protein * 4) - (fat * 9)) / 4);
```

Every macronutrient has a fixed calorie-per-gram value: Protein = 4 kcal/g, Carbohydrates = 4 kcal/g, Fat = 9 kcal/g. These are universal nutritional constants.

- Protein: 2g per kg of bodyweight is the standard recommendation for active individuals and those building muscle. This ensures enough protein for muscle repair and growth.
- Fat: 25% of total calories come from fat. Fat is essential for hormones and nutrient absorption. Each gram of fat provides 9 calories, so we divide the fat calories by 9 to get grams.
- Carbs: Whatever calories remain after accounting for protein and fat go to carbohydrates. Carbs are the flexible macro that fills the remaining calorie gap.

3.11 Displaying Results — innerHTML and style.display

```
let resultSection = document.getElementById("result-section");
resultSection.style.display = "block";

document.getElementById("greetings").innerHTML =
    "Hello " + name + "! Here are your personalized Results";

document.getElementById("bmi-result").innerHTML =
    "Your BMI: <strong>" + bmi + "</strong>";
```

The results section starts with style="display:none" in HTML, making it invisible. When JavaScript sets display to "block", it becomes visible. This is dynamic styling — JavaScript changing CSS in real time.

innerHTML lets JavaScript set the content inside any HTML element. It accepts plain text and also real HTML tags like for bold text. This is how the calculated values are written directly onto the page without refreshing.

3.12 Meal Logger — Arrays and Objects

```
let meals = [];

let meal = {
    name:     mealName,
    calories: mealCalories,
    protein:  mealProtein,
    carbs:    mealCarbs,
    fat:      mealFat
};
meals.push(meal);
```

An array (meals = []) is a list that can hold multiple items. Each time the user adds a meal, a meal object is created with all the food details grouped together, and then pushed into the array. An object groups related values under one name — instead of 5 separate variables for each meal, one object holds everything neatly.

The push() method adds a new item to the end of the array. This is how the app remembers all meals — they are stored in the meals array so the delete function can find and remove them by their index position.

3.13 Creating Table Rows Dynamically — createElement and appendChild

```
let tbody = document.getElementById("meal-tbody");
let row   = document.createElement("tr");
row.id    = "meal-row-" + index;
```

```
row.innerHTML =
    "<td>" + meal.name + "</td>" +
    "<td>" + meal.calories + "</td>" +
    "<td><button onclick='deleteMeal(" + index + ")'>Delete</button></td>";
tbody.appendChild(row);
```

createElement("tr") creates a brand new HTML table row element entirely in JavaScript. We assign it an id like "meal-row-0", "meal-row-1" etc. so we can find and remove it later. innerHTML fills the row with the meal data and a Delete button. appendChild attaches the new row to the table body — it appears on screen instantly. This is advanced DOM manipulation.

3.14 Deleting a Meal

```
function deleteMeal(index) {
    totalCalories -= meals[index].calories;
    totalProtein  -= meals[index].protein;
    totalCarbs    -= meals[index].carbs;
    totalFat      -= meals[index].fat;
    meals[index] = null;
    document.getElementById("meal-row-" + index).remove();
    updateTotals();
}
```

The -= operator subtracts the deleted meal's values from the running totals. meals[index] = null empties that slot in the array. remove() deletes the HTML row from the page. updateTotals() is then called to refresh all displays. This shows a real-world use of arrays — storing data in indexed positions so items can be found and removed by position.

3.15 Over/Under Target Warning — className

```
let warningBox = document.getElementById("calorie-warning");
let calorieBalance = targetCalorie - totalCalories;

if (totalCalories === 0) {
    warningBox.innerHTML = "";
    warningBox.className = "";
} else if (calorieBalance < -100) {
    warningBox.innerHTML = "⚠️ Exceeded target by " + Math.abs(calorieBalance) + "
kcal";
    warningBox.className = "warning-over";
} else if (calorieBalance > 100) {
    warningBox.innerHTML = "✅ Within target! " + calorieBalance + " kcal
remaining";
    warningBox.className = "warning-under";
} else {
    warningBox.innerHTML = "🎯 Perfect! Daily calorie target hit!";
    warningBox.className = "warning-exact";
}
```

className lets JavaScript change the CSS class of an element dynamically. Each class (warning-over, warning-under, warning-exact) has a different background and text colour defined in CSS. By swapping the className, the warning box changes its entire appearance in real time. Math.abs() converts a negative number to positive so the message reads correctly.

3.16 Reset Button — confirm() and Resetting State

```
let confirmed = confirm("Are you sure you want to reset everything?");
if (confirmed === false) { return; }

document.getElementById("meal-tbody").innerHTML = "";
document.getElementById("result-section").style.display = "none";
meals = [];
totalCalories = 0;
```

confirm() shows a browser dialog with OK and Cancel buttons. It returns true if OK is clicked and false if Cancel is clicked. Setting innerHTML = "" completely wipes all rows from the meal table. Setting meals = [] empties the array. Setting display back to none hides the results section. This shows how JavaScript can fully reset an app's state without refreshing the page.

3.17 addMealToTable() — Building a Table Row with JavaScript

This function runs every time the user clicks Add Meal. Its job is to take the meal data and create a new visible row in the Logged Meals table on screen. It receives two things: the meal object (containing all the food details) and the index (the position number of this meal in the meals array).

```
function addMealToTable(meal, index) {

    // Grabbing Table Body where row goes
    let tbody = document.getElementById("meal-tbody");

    // Create a new table row element
    let row = document.createElement("tr");

    // Give row an id so we can find and delete it later
    row.id = "meal-row-" + index;

    // Fill the row with meal data & adding delete button
    row.innerHTML =
        "<td>" + meal.name     + "</td>" +
        "<td>" + meal.calories + "</td>" +
        "<td>" + meal.protein  + "g</td>" +
        "<td>" + meal.carbs    + "g</td>" +
        "<td>" + meal.fat      + "g</td>" +
```

```
        "<td><button class='delete-btn' onclick='deleteMeal(" + index +
")'>Delete</button></td>";

    // Add row to the table
    tbody.appendChild(row);
}
```

Step 1 — document.getElementById("meal-tbody") finds the table body in the HTML where all meal rows live. Think of tbody as the container that holds all the rows.

Step 2 — document.createElement("tr") creates a brand new table row entirely in JavaScript. At this point the row exists in memory but is not visible on screen yet.

Step 3 — row.id = "meal-row-" + index gives the row a unique id. The first meal gets id "meal-row-0", the second gets "meal-row-1" and so on. This unique id is essential — it is how the Delete button later knows exactly which row to remove.

Step 4 — row.innerHTML fills the row with all the meal data wrapped in table cell tags (<td>). It also adds a red Delete button at the end. The onclick inside the button passes the index number so deleteMeal() knows which meal was clicked.

Step 5 — tbody.appendChild(row) attaches the finished row to the table. The moment this line runs, the new row appears on screen instantly — without any page refresh. This is DOM manipulation in action.

3.18 updateTotals() — Refreshing All Displays After Every Change

This function is the "refresh engine" of the app. It is called every time a meal is added or deleted. Its job is to update every number shown on screen — the daily totals, the remaining targets, and the colour-coded warning message.

```
function updateTotals() {

    // Update total displays
    document.getElementById("total-calories-display").innerHTML =
        "Total Calories: <strong>" + totalCalories.toFixed(1) + " Kcal</strong>";
    document.getElementById("total-protein-display").innerHTML =
        "Total Protein: <strong>" + totalProtein.toFixed(1) + " g</strong>";
    document.getElementById("total-carbs-display").innerHTML =
        "Total Carbs: <strong>" + totalCarbs.toFixed(1) + " g</strong>";
    document.getElementById("total-fat-display").innerHTML =
        "Total Fat: <strong>" + totalFat.toFixed(1) + " g</strong>";

    // Calculate remaining targets
    let remainingCalories = targetCalorie - totalCalories;
```

```
    let remainingProtein  = protein - totalProtein;
    let remainingCarbs    = carbs   - totalCarbs;
    let remainingFat      = fat     - totalFat;

    // Update Remaining displays
    document.getElementById("remaining-calories-display").innerHTML =
        "Calories Remaining: <strong>" + remainingCalories.toFixed(1) + "
kcal</strong>";
    document.getElementById("remaining-protein-display").innerHTML =
        "Protein Remaining: <strong>" + remainingProtein.toFixed(1) + "g</strong>";
    document.getElementById("remaining-carbs-display").innerHTML =
        "Carbs Remaining: <strong>" + remainingCarbs.toFixed(1) + "g</strong>";
    document.getElementById("remaining-fat-display").innerHTML =
        "Fat Remaining: <strong>" + remainingFat.toFixed(1) + "g</strong>";

    // Over & Under Target Warning
    let warningBox    = document.getElementById("calorie-warning");
    let calorieBalance = targetCalorie - totalCalories;

    if (totalCalories === 0) {
        warningBox.innerHTML = "";
        warningBox.className = "";
    } else if (calorieBalance < -100) {
        warningBox.innerHTML = "⚠️ You have exceeded your calorie target by "
            + Math.abs(calorieBalance).toFixed(0) + " kcal! Consider lighter
Meals!";
        warningBox.className = "warning-over";
    } else if (calorieBalance > 100) {
        warningBox.innerHTML = "✅ You are within your target! "
            + Math.abs(calorieBalance).toFixed(0) + " kcal remaining for today!";
        warningBox.className = "warning-under";
    } else {
        warningBox.innerHTML = "🎯 Perfect! You have hit your daily calorie
target!";
        warningBox.className = "warning-exact";
    }
}
```

Part 1 — Updating Daily Totals: The first four lines find each display paragraph by its id and overwrite its content with the latest running total. toFixed(1) ensures numbers always show one decimal place (e.g. 1000.0) instead of long decimals.

Part 2 — Calculating Remaining: These four lines do simple subtraction — target minus what has been eaten so far. For example, if the target is 2501 and 1000 has been consumed, remainingCalories becomes 1501. If the user has eaten more than the target, remaining becomes a negative number.

Part 3 — Updating Remaining Displays: Same as Part 1 — the remaining values are written into their display paragraphs on screen.

Part 4 — Over/Under Warning: calorieBalance is calculated (target minus total consumed). Then an if/else if/else chain decides which message and colour to show. warningBox.className swaps the CSS class on the warning box — each class has a different background colour defined in the CSS. Math.abs() converts negative numbers to positive so the message reads correctly (e.g. "exceeded by 499 kcal" instead of "exceeded by -499 kcal").

3.19 deleteMeal() — Removing a Meal and Updating Everything

This function runs when the user clicks the red Delete button on any meal row. The index parameter tells the function exactly which meal to delete — for example, index 0 means the first meal, index 1 means the second meal.

```
function deleteMeal(index) {

    // Subtract this meal's values from running totals
    totalCalories -= meals[index].calories;
    totalProtein  -= meals[index].protein;
    totalCarbs    -= meals[index].carbs;
    totalFat      -= meals[index].fat;

    // Remove the meal from our array
    meals[index] = null;

    // Remove the row from the table
    let row = document.getElementById("meal-row-" + index);
    row.remove();

    // Update Totals Display
    updateTotals();
}
```

Step 1 — Subtract from totals: meals[index] means "go to the meals array and find the meal at this position". Then .calories, .protein, .carbs, .fat access that meal's stored values. The -= operator subtracts each value from the running totals. For example, if totalCalories was 1500 and the deleted meal had 500 calories, totalCalories becomes 1000.

Step 2 — meals[index] = null: This empties that slot in the array by setting it to null (meaning nothing). The reason we do not completely remove the item is to keep all other meal indexes intact. If the array shifted after a deletion, the Delete buttons of other meals would point to the wrong index and break. Setting to null is safer — it clears the data while preserving the positions of all other meals.

Step 3 — Remove the HTML row: document.getElementById("meal-row-" + index) builds the id of the row to delete. If index is 1, it looks for "meal-row-1". The .remove() method then deletes that row from the table completely. The user sees it disappear from the screen instantly.

Step 4 — updateTotals(): After deleting, all the totals and the warning message on screen need to refresh. Rather than rewriting all that code again, we simply call the updateTotals() function that already handles it. This is a key principle of programming — write a function once and reuse it wherever needed.

4. Reflection — Challenges & Lessons Learned

4.1 Challenge 1: Global vs Local Variables (Biggest Challenge)

The most significant challenge during development was understanding the difference between global and local variables in JavaScript. This caused several hours of debugging where results were showing as 0 even when data was entered correctly.

The problem: targetCalorie, protein, carbs, and fat were initially declared using let inside the calculate button's event listener. This made them local variables — meaning they only existed inside that function. When the updateTotals function tried to use these variables, it could not find them and JavaScript treated them as 0.

```
// WRONG - declared inside the function (local)
calbtn.addEventListener("click", function() {
    let targetCalorie = dailyCalories - 500;  // dies here
});

function updateTotals() {
    let remaining = targetCalorie - total;  // ERROR: targetCalorie is 0!
}
```

The fix was to declare these variables at the very top of the script, outside all functions:

```
// CORRECT - declared globally (accessible everywhere)
let targetCalorie = 0;
let protein = 0;
let carbs   = 0;
let fat     = 0;
```

Then inside the calculate listener, the let keyword was removed when assigning these variables — so they update the global ones rather than creating new local ones.

Lesson learned: Variables declared inside a function (using let) only exist inside that function. If multiple functions need to share data, those variables must be declared globally at the top of the script. This concept is called variable scope and is one of the most important concepts in JavaScript.

4.2 Challenge 2: Wrong Comment Syntax

HTML, CSS, and JavaScript all use different comment styles. Early in the project, HTML was written with the wrong comment format:

```
<!Name input>     ← WRONG HTML comment
/* Name input */  ← CSS comment style (also wrong in HTML)
<!-- Name input --> ← CORRECT HTML comment
```

This caused browser warnings. The correct comment syntax for each language is:

- HTML: <!-- comment -->
- CSS: /* comment */
- JavaScript: // single line comment

4.3 Challenge 3: Warning Code Placed Outside Function

The over/under target warning code was initially placed outside the updateTotals() function. This meant it only ran once when the page first loaded (when no meals existed) and never again. Moving the warning code inside updateTotals() meant it ran every time a meal was added or deleted — producing the correct live behaviour.

Lesson learned: Code that should respond to user actions must be inside a function that is called when those actions happen. Code placed at the top level of a script runs only once when the page loads.

4.4 Key JavaScript Concepts Learned Through This Project

Variables (let)	Storing data like weight, age, calories in named boxes
Functions	Reusable blocks of code — addMealToTable(), deleteMeal(), updateTotals()
Event Listeners	Detecting button clicks and running code in response
DOM Manipulation	Finding HTML elements and changing their content and style
if / else if / else	Making decisions based on data values
Arrays	Storing lists of meals that can be added to and removed from
Objects	Grouping related meal data (name, calories, protein, carbs, fat) together
parseFloat()	Converting text input values into real numbers for calculations
Math.round()	Rounding decimal numbers to whole numbers
Global vs Local Scope	Understanding which variables are accessible where in the code

4.6 Ideas for Future Improvements

- Add a local storage feature so meal logs are saved even after the browser is closed
- Add a visual progress bar showing how close the user is to their calorie target
- Add a food database so users can search for common foods instead of entering macros manually

- Add the ability to save and compare daily logs across multiple days
- Make the app mobile-responsive with a better layout for small screens

Appendix A: Full HTML/CSS/JavaScript Code

The complete code of the application is presented below. All HTML structure, CSS styling, and JavaScript logic are contained in a single file: index.html

```
<!DOCTYPE HTML>
<html lang="en"> <!-- Tells the browser page is in English -->
<head>
    <meta charset="UTF-8"/> <!-- Ensures special characters display correctly -->
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Fitness & Nutrition Tracker</title>
    <style>
        body { font-family: Tahoma, sans-serif; background-color: #f0f4f8;
               padding: 20px; color: #333333; }
        h1 { text-align: center; color: #2c7a4b; }
        #input-section { background-color: #ffffff; max-width: 600px;
                         margin: 0 auto; padding: 30px; border-radius: 12px; }
        label { display: block; margin-top: 15px; font-weight: bold; }
        input, select { width: 100%; padding: 8px; margin-top: 5px;
                        border: 1px solid #cccccc; border-radius: 6px;
                        font-family: Tahoma, sans-serif; box-sizing: border-box; }
        #cal-btn { display: block; width: 100%; margin-top: 20px; padding: 10px;
                   background-color: #2c7a4b; color: #ffffff; font-size: 16px;
                   border: none; border-radius: 8px; cursor: pointer; }
        #cal-btn:hover { background-color: #235f3a; }
        #result-section { background-color: #ffffff; max-width: 600px;
                          margin: 20px auto; padding: 30px; border-radius: 12px; }
        h3 { color: #2c7a4b; margin-top: 20px; border-bottom: 1px solid #cccccc; }
        #meal-btn { display: block; width: 100%; margin-top: 20px; padding: 10px;
                    background-color: #3a86ff; color: #ffffff; font-size: 16px;
                    border-radius: 8px; cursor: pointer; }
        #meal-table { width: 100%; border-collapse: collapse; margin-top: 15px; }
        #meal-table th, #meal-table td { border: 1px solid #cccccc;
                                         padding: 8px; text-align: center; }
        #meal-section { background-color: #ffffff; max-width: 600px;
                        margin: 20px auto; padding: 30px; border-radius: 12px; }
        .delete-btn { background-color: #e63946; color: white; border: none;
                      padding: 5px 10px; border-radius: 4px; cursor: pointer; }
        #reset-btn { display: block; width: 100%; margin-top: 10px; padding: 10px;
                     background-color: #aaaaaa; color: #ffffff; font-size: 16px;
                     border: none; border-radius: 8px; cursor: pointer; }
        .warning-over  { background-color: #ffe0e0; color: #c1121f; padding: 10px;
                         border-radius: 6px; font-weight: bold; margin-top: 10px; }
        .warning-under { background-color: #e0ffe0; color: #2c7a4b; padding: 10px;
                         border-radius: 6px; font-weight: bold; margin-top: 10px; }
        .warning-exact { background-color: #fff8e0; color: #e6a817; padding: 10px;
```

```
                        border-radius: 6px; font-weight: bold; margin-top: 10px; }
    </style>
</head>
<body>
    <h1>Fitness & Nutrition Tracker</h1>
    <div id="input-section">
        <h2>Enter Details</h2>
        <label for="name">Your Name:</label>
        <input type="text" id="name">
        <label for="age">Age:</label>
        <input type="number" id="age">
        <label for="gender">Gender:</label>
        <select id="gender">
            <option value="">--Select Gender--</option>
            <option value="male">Male</option>
            <option value="female">Female</option>
        </select>
        <label for="weight">Weight in KG:</label>
        <input type="number" id="weight">
        <label for="height">Height in cm:</label>
        <input type="number" id="height">
        <label for="activity">Activity Level:</label>
        <select id="activity">
            <option value="">--Select Activity Level--</option>
            <option value="1.2">Sedentary (Little or No Exercise)</option>
            <option value="1.375">Lightly Active (1-3 Days/week)</option>
            <option value="1.55">Moderately Active (3-5 Days/week)</option>
            <option value="1.725">Very Active (6-7 Days/week)</option>
        </select>
        <label for="goal">Your Goal:</label>
        <select id="goal">
            <option value="">--Select Your Goal--</option>
            <option value="loss">Weight Loss</option>
            <option value="maintain">Maintain Weight</option>
            <option value="gain">Gain Muscle</option>
        </select>
        <button id="cal-btn">Calculate</button>
        <button id="reset-btn">Reset Everything</button>
    </div>

    <!-- Result Section - Hidden by default -->
    <div id="result-section" style="display:none">
        <h2>Your Results</h2>
        <p id="greetings"></p>
        <h3>BMI Results</h3>
        <p id="bmi-result"></p>
        <p id="bmi-category"></p>
        <h3>Daily Calories</h3>
        <p id="bmr-result"></p>
        <p id="dailyCalories-result"></p>
        <p id="targetCalories-result"></p>
        <h3>Daily Macronutrients</h3>
        <p id="protein-result"></p>
```

```
        <p id="carbs-result"></p>
        <p id="fat-result"></p>
    </div>

    <!-- Meal Logger Section - Hidden by default -->
    <div id="meal-section" style="display:none">
        <h2>Daily Meal Logger</h2>
        <label for="meal-name">Food Item:</label>
        <input type="text" id="meal-name" placeholder="E.g. Chicken, Lentils"/>
        <label for="meal-calories">Calories Kcal:</label>
        <input type="number" id="meal-calories"/>
        <label for="meal-protein">Protein (g):</label>
        <input type="number" id="meal-protein"/>
        <label for="meal-carbs">Carbs (g):</label>
        <input type="number" id="meal-carbs"/>
        <label for="meal-fat">Fat (g):</label>
        <input type="number" id="meal-fat"/>
        <button id="meal-btn">Add Meal</button>
        <h3>Logged Meals</h3>
        <table id="meal-table">
            <thead><tr>
                <th>Food</th><th>Calories</th><th>Protein</th>
                <th>Carbs</th><th>Fat</th><th>Action</th>
            </tr></thead>
            <tbody id="meal-tbody"></tbody>
        </table>
        <h3>Daily Totals</h3>
        <p id="total-calories-display"></p>
        <p id="total-protein-display"></p>
        <p id="total-carbs-display"></p>
        <p id="total-fat-display"></p>
        <h3>Remaining to Hit your Target</h3>
        <p id="remaining-calories-display"></p>
        <p id="remaining-protein-display"></p>
        <p id="remaining-carbs-display"></p>
        <p id="remaining-fat-display"></p>
        <p id="calorie-warning"></p>
    </div>

    <script>
        // Global Variables - accessible by all functions
        let targetCalorie = 0;
        let protein = 0;
        let carbs   = 0;
        let fat     = 0;

        // Grab the Calculate button
        let calbtn = document.getElementById("cal-btn");

        calbtn.addEventListener("click", function() {
            // Read all input values
            let name     = document.getElementById("name").value;
```

```
let age      = document.getElementById("age").value;
let gender   = document.getElementById("gender").value;
let weight   = document.getElementById("weight").value;
let height   = document.getElementById("height").value;
let activity = document.getElementById("activity").value;
let goal     = document.getElementById("goal").value;

// Validation - check all fields are filled
if(name===""||age===""||weight===""||height===""
   ||gender===""||activity===""||goal===""){
    alert("Please fill in the Mandatory Details");
    return;
}
if(age<=0 || height<=0 || weight<=0){
    alert("Age, Weight & Height must be greater than zero");
    return;
}

// Convert to numbers
age    = parseFloat(age);
weight = parseFloat(weight);
height = parseFloat(height);

// BMI Calculation
let heightMtrs = height / 100;
let bmi = weight / (heightMtrs * heightMtrs);
bmi = bmi.toFixed(1);

// BMI Category
let bmiCategory;
if(bmi < 18.5)      bmiCategory = "Underweight";
else if(bmi < 25)   bmiCategory = "Normal Weight";
else if(bmi < 30)   bmiCategory = "Overweight";
else                bmiCategory = "Obese";

// BMR - Mifflin-St Jeor Formula
let bmr;
if(gender === "male")
    bmr = (10*weight) + (6.25*height) - (5*age) + 5;
else
    bmr = (10*weight) + (6.25*height) - (5*age) - 161;
bmr = Math.round(bmr);

// Daily calories based on activity
let activityMulti = parseFloat(activity);
let dailyCalories = Math.round(bmr * activityMulti);

// Target calories based on goal
if(goal === "loss")         targetCalorie = dailyCalories - 500;
else if(goal === "maintain") targetCalorie = dailyCalories;
else                         targetCalorie = dailyCalories + 500;
```

```
            // Macronutrients
            protein = Math.round(weight * 2);
            fat     = Math.round((targetCalorie * 0.25) / 9);
            carbs   = Math.round((targetCalorie-(protein*4)-(fat*9)) / 4);

            // Show results section
            document.getElementById("result-section").style.display = "block";

            // Fill result paragraphs
            document.getElementById("greetings").innerHTML =
                "Hello " + name + "! Here are your personalized Results";
            document.getElementById("bmi-result").innerHTML =
                "Your BMI: <strong>" + bmi + "</strong>";
            document.getElementById("bmi-category").innerHTML =
                "Category: <strong>" + bmiCategory + "</strong>";
            document.getElementById("bmr-result").innerHTML =
                "Your BMR (Calories at rest): <strong>" + bmr + " kcal</strong>";
            document.getElementById("dailyCalories-result").innerHTML =
                "Daily Calorie Burn: <strong>" + dailyCalories + " kcal</strong>";
            document.getElementById("targetCalories-result").innerHTML =
                "Target Calories (" + goal + "): <strong>" + targetCalorie + "
kcal</strong>";
            document.getElementById("protein-result").innerHTML =
                "Protein: <strong>" + protein + "g</strong> per day";
            document.getElementById("carbs-result").innerHTML =
                "Carbs: <strong>" + carbs + "g</strong> per day";
            document.getElementById("fat-result").innerHTML =
                "Fat: <strong>" + fat + "g</strong> per day";

            // Show meal logger section
            document.getElementById("meal-section").style.display = "block";
        });

        // Meal Logger
        let meals = [];
        let totalCalories = 0;
        let totalProtein  = 0;
        let totalCarbs    = 0;
        let totalFat      = 0;

        let mealBtn = document.getElementById("meal-btn");

        mealBtn.addEventListener("click", function() {
            let mealName     = document.getElementById("meal-name").value;
            let mealCalories = parseFloat(document.getElementById("meal-
calories").value);
            let mealProtein  = parseFloat(document.getElementById("meal-
protein").value);
            let mealCarbs    = parseFloat(document.getElementById("meal-
carbs").value);
```

```
            let mealFat      = parseFloat(document.getElementById("meal-
fat").value);

            if(mealName === ""){
                alert("Please enter a Food Name"); return;
            }

if(isNaN(mealCalories)||isNaN(mealProtein)||isNaN(mealCarbs)||isNaN(mealFat)){
                alert("Please Enter Valid Numbers for Nutrition Values"); return;
            }

            let meal = { name:mealName, calories:mealCalories,
                         protein:mealProtein, carbs:mealCarbs, fat:mealFat };
            meals.push(meal);

            totalCalories += mealCalories;
            totalProtein  += mealProtein;
            totalCarbs    += mealCarbs;
            totalFat      += mealFat;

            addMealToTable(meal, meals.length - 1);
            updateTotals();

            document.getElementById("meal-name").value     = "";
            document.getElementById("meal-calories").value = "";
            document.getElementById("meal-protein").value  = "";
            document.getElementById("meal-carbs").value    = "";
            document.getElementById("meal-fat").value      = "";
        });

        function addMealToTable(meal, index) {
            let tbody = document.getElementById("meal-tbody");
            let row   = document.createElement("tr");
            row.id    = "meal-row-" + index;
            row.innerHTML =
                "<td>" + meal.name     + "</td>" +
                "<td>" + meal.calories + "</td>" +
                "<td>" + meal.protein  + "g</td>" +
                "<td>" + meal.carbs    + "g</td>" +
                "<td>" + meal.fat      + "g</td>" +
                "<td><button class='delete-btn' onclick='deleteMeal(" + index +
")'>
                Delete</button></td>";
            tbody.appendChild(row);
        }

        function deleteMeal(index) {
            totalCalories -= meals[index].calories;
            totalProtein  -= meals[index].protein;
            totalCarbs    -= meals[index].carbs;
            totalFat      -= meals[index].fat;
            meals[index]   = null;
```

```
            document.getElementById("meal-row-" + index).remove();
            updateTotals();
        }

        function updateTotals() {
            document.getElementById("total-calories-display").innerHTML =
                "Total Calories: <strong>" + totalCalories.toFixed(1) + "
Kcal</strong>";
            document.getElementById("total-protein-display").innerHTML =
                "Total Protein: <strong>" + totalProtein.toFixed(1)  + "
g</strong>";
            document.getElementById("total-carbs-display").innerHTML =
                "Total Carbs: <strong>"   + totalCarbs.toFixed(1)    + "
g</strong>";
            document.getElementById("total-fat-display").innerHTML =
                "Total Fat: <strong>"     + totalFat.toFixed(1)      + "
g</strong>";

            let remainingCalories = targetCalorie - totalCalories;
            let remainingProtein  = protein       - totalProtein;
            let remainingCarbs    = carbs         - totalCarbs;
            let remainingFat      = fat           - totalFat;

            document.getElementById("remaining-calories-display").innerHTML =
                "Calories Remaining: <strong>" + remainingCalories.toFixed(1) + "
kcal</strong>";
            document.getElementById("remaining-protein-display").innerHTML =
                "Protein Remaining: <strong>" + remainingProtein.toFixed(1) +
"g</strong>";
            document.getElementById("remaining-carbs-display").innerHTML =
                "Carbs Remaining: <strong>" + remainingCarbs.toFixed(1) +
"g</strong>";
            document.getElementById("remaining-fat-display").innerHTML =
                "Fat Remaining: <strong>" + remainingFat.toFixed(1) + "g</strong>";

            // Over/Under Warning
            let warningBox     = document.getElementById("calorie-warning");
            let calorieBalance = targetCalorie - totalCalories;

            if(totalCalories === 0) {
                warningBox.innerHTML = ""; warningBox.className = "";
            } else if(calorieBalance < -100) {
                warningBox.innerHTML = "Over target by " +
Math.abs(calorieBalance).toFixed(0) + " kcal!";
                warningBox.className = "warning-over";
            } else if(calorieBalance > 100) {
                warningBox.innerHTML = "Within target! " +
calorieBalance.toFixed(0) + " kcal remaining!";
                warningBox.className = "warning-under";
            } else {
                warningBox.innerHTML = "Perfect! Daily calorie target hit!";
                warningBox.className = "warning-exact";
            }
```

```
        }

        // Reset Button
        let resetBtn = document.getElementById("reset-btn");
        resetBtn.addEventListener("click", function() {
            let confirmed = confirm("Are you sure you want to reset everything?");
            if(confirmed === false) return;

            document.getElementById("name").value     = "";
            document.getElementById("age").value      = "";
            document.getElementById("gender").value   = "";
            document.getElementById("height").value   = "";
            document.getElementById("weight").value   = "";
            document.getElementById("activity").value = "";
            document.getElementById("goal").value     = "";

            document.getElementById("result-section").style.display = "none";
            document.getElementById("meal-section").style.display   = "none";

            targetCalorie = 0; protein = 0; carbs = 0; fat = 0;
            totalCalories = 0; totalProtein = 0; totalCarbs = 0; totalFat = 0;
            meals = [];

            document.getElementById("meal-tbody").innerHTML              = "";
            document.getElementById("total-calories-display").innerHTML  = "";
            document.getElementById("total-protein-display").innerHTML   = "";
            document.getElementById("total-carbs-display").innerHTML     = "";
            document.getElementById("total-fat-display").innerHTML       = "";
            document.getElementById("remaining-calories-display").innerHTML = "";
            document.getElementById("remaining-protein-display").innerHTML  = "";
            document.getElementById("remaining-carbs-display").innerHTML    = "";
            document.getElementById("remaining-fat-display").innerHTML      = "";
            document.getElementById("calorie-warning").innerHTML            = "";
            document.getElementById("calorie-warning").className            = "";
        });
    </script>
</body>
</html>
```